

Unit - 2

Context free language

* Context Free Grammar :-

The context free language use Context free grammar that can be formally defined as

$$G = (V, T, P, S) \text{ where}$$

V = finite set of variables

T = finite set of terminals

P = Set of Production function

S = Starting symbol

Note variables are denoted by capital letters always (Non-terminals)

Small letter are used to denote terminals.

• Variables - Non-terminals.

- Used to represent intermediate steps in derivation

- Denoted by Capital letters.

$$G \Rightarrow V \rightarrow Aa$$

• Terminals - These form actual string of the language. Cannot be replaced further.

$$G \Rightarrow V \rightarrow Aa \text{ terminal}$$

Always present at the right side of any production rule

- Production Rules - Finite set of rules of the form,

$$A \rightarrow a$$

where $A \in V$

$$a \in (V \cup T)^*$$

means left side $\rightarrow$ single variable.

Right side $\rightarrow$ Any combination of variable + terminals.

- Start Symbol - A special variable from which derivation begins.

$$S \in V$$

Note In Right side of production rules can be Terminal, variable, empty (ϵ) also.

$$G = (\{S\}, \{a, b\}, R, S)$$

where rules are

$$S \rightarrow aSb \mid \epsilon$$

Examples of Context-free Grammar:-

Q Construct all CFN for the language having any number of 'a's over the set $\Sigma = \{a\}$

Solⁿ Regular Expression $\Rightarrow$

$$RE = a^*$$

Let string = "aaa"

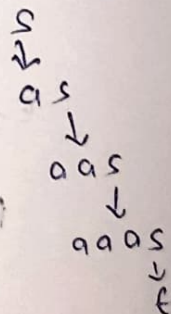
Production Rules

$$S \rightarrow aS$$

Rule 1

$$S \rightarrow \epsilon$$

Rule 2



Ans (aaa)

Ex: Try to recognize the language L for given CFG

$G = [\{S\}, \{a, b\}, P, \{S\}]$ where

$$P = \{ S \rightarrow asb \mid ab \}$$

Sol: $P = \{ S \rightarrow asb \mid ab \}$ means

$$S \rightarrow asb$$

$$S \rightarrow ab$$

as it follows

S

$\downarrow$

asb

$\downarrow$

$aaSbb$

$\downarrow$

$aaasbbb$

$\downarrow$

$aaaa\underline{ab}bbb$

that means any number of a
at first we can get by
these rules.

if we put $S = ab$

number of b 's are equal to
number of a 's

Hence

$$\{ L = a^n b^n \text{ where } n \geq 1 \}$$

Ex: Construct the CFG for the $R \in (0+1)^*$

Sol:

$$P = \left\{ \begin{array}{l} S \rightarrow 0S \\ S \rightarrow 1S \\ S \rightarrow \epsilon \end{array} \right\}$$

Ex: Construct a grammar for the language containing
string of at least two a 's

Sol:

$$\text{Let } P = \left\{ \begin{array}{l} S \rightarrow AaAaA \\ A \rightarrow aA \mid bA \mid \epsilon \end{array} \right\}$$

as 'a' can be in middle / at start / at end

So set two a 's by default that can add
more a 's or b 's.

Q Construct a grammar generating $L = w.c.w^T$
 where $w = \{a, b\}^*$

Solⁿ $w = \text{any string of a's \& b's}$ $\rightarrow (w^T = \text{Reverse of } w)$
 $w^T = \text{Reverse of } w$
 $c \text{ at middle.}$

So, CFG rule.

if $w = \epsilon \rightarrow c$

$w = a \rightarrow aca$

$w = ab \rightarrow abcba$

$w = ba \rightarrow bacab$

(mirror around c)

$S \rightarrow asa$

$S \rightarrow bsb$

$S \rightarrow c$

Let check for $w = aqbb$

S
 $\downarrow a$

asa

$\downarrow a$

$asasa$

$\downarrow b$

$abbsbaa$

$\downarrow b$

$aabbbsbbbaa$

$\downarrow$

$aabbcbbaa$

Ans

Difference between Regular & Context free language :-

Features	Regular language	Context free language
Position in hierarchy	lowest level, simple structure	Higher than Regular language & Nested stack
Grammar type	Regular grammar	Context free grammar
Recognised by	Finite Automata	Push Down Automata
Memory	No memory used because it shows only current state	Stack memory is used
Powerful	Low powerful	High powerful
Example	a^*b^* , a^*b	$a^n b^n$, $a^{2n} b^n$
Application	Text search Lexical search Pattern matching	Syntax Analysis (parsing) NLP

Note error { Lexical error (spelling mistake)
Syntax (Grammatical) error
Semantic (Out of bound like divide by zero) error

All regular languages are context free languages but it is not compulsory that all context free languages are regular language.

$RL \subset CFL$ but $CFL \not\subset RL$

* Derivation & Languages: The production Rules are used to derive certain strings. The generation of language using specific Rules is called 'derivation'.

The language generated by 'G', Grammar is denoted by $L(G)$.

Notations

$\Rightarrow$ $\rightarrow$ one step derivation

$\Rightarrow^*$ $\rightarrow$ multiple steps

$G \equiv S \rightarrow asb|c$

String 'aabb' $\Rightarrow S \Rightarrow asb$
 $\Rightarrow aasbb$
 $\Rightarrow aabb$

At every step, leftmost S is replaced.

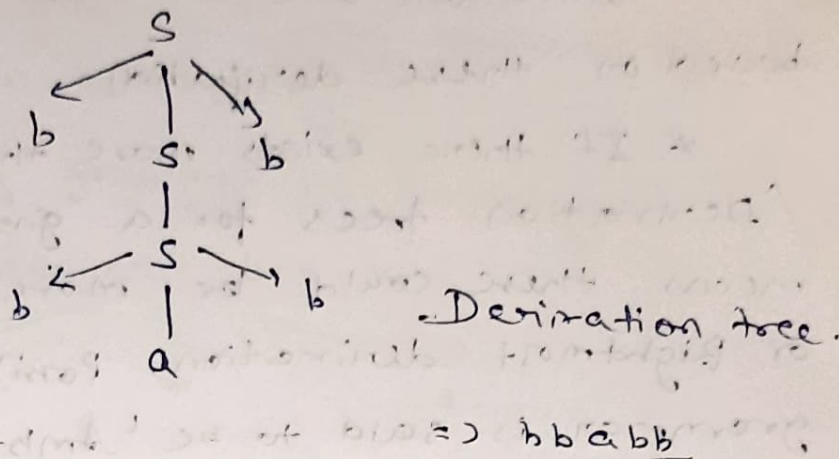
* Relationship between Derivation & Derivation Tree

Derivation tree: It is a graphical representation for the derivation of the given production rules for a given GFG.

properties

- (i) Root node is always a node indicating start symbol.
- (ii) Derivation is read from left to right.
- (iii) Leaf nodes always terminal nodes.
- (iv) Intermediate node always the non terminal

Ex. $S \rightarrow bsb$ is a production rule. The 'S' is a start symbol.
 $S \rightarrow a$
 $S \rightarrow b$



* LMDT (Left most Derivation tree) & RMDT (Right most derivation tree):

Derivation means replacement of non-terminal by its appropriate definition, or rules.

Methods of Derivation

* Left Most Derivation Tree

Leftmost non-terminal is replaced first

$S \Rightarrow aSb$
 $aSbb$
 $aabb$

Used in Top-down parsing

* Right most derivation Tree

Right most non-terminal is replaced first

$S \Rightarrow aSb$
 $aSbb$
 $aabb$

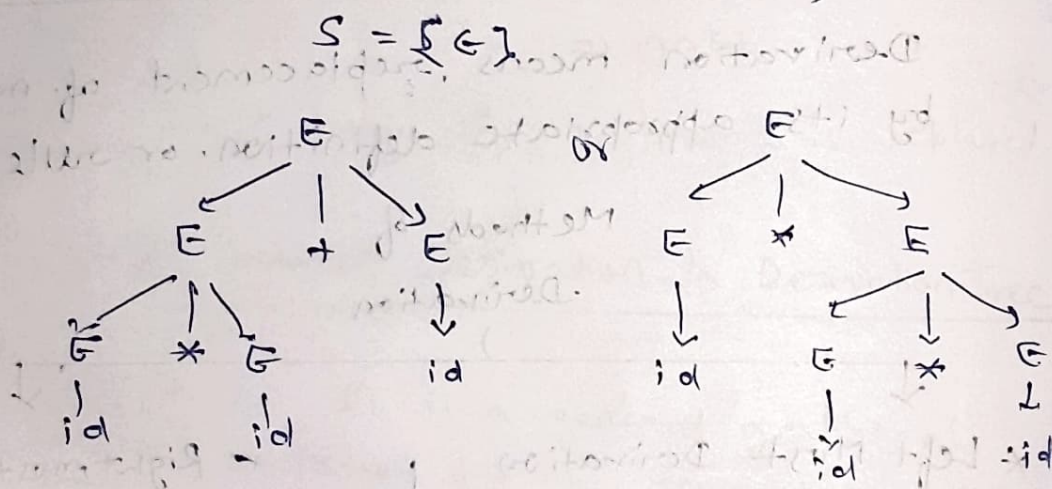
Used in Bottom-up parsing.

* Ambiguity :- The grammar can be derived in either leftmost derivation or rightmost derivation. We can draw a derivation tree based on these derivation.

* If there exists more than one parse tree / Derivation trees for a given grammar, that means there could be more than one leftmost or Rightmost derivation. Possible (p) then that grammar is said to be 'Ambiguous grammar'

$$G = (V, T, P, S)$$

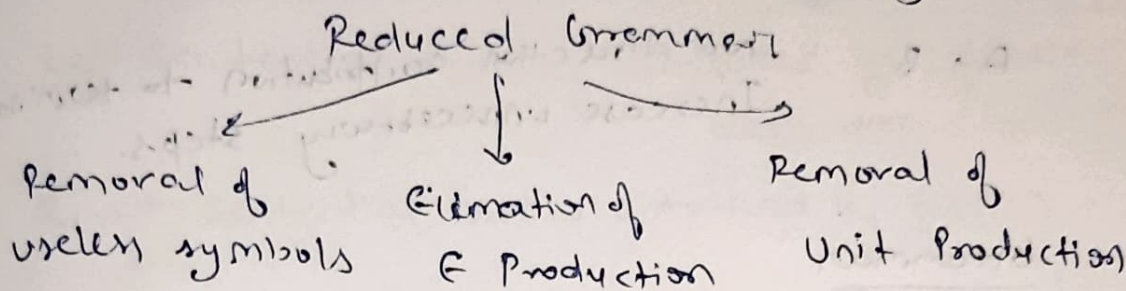
$$V = \{E\}, T = \{id\}, P = \left\{ \begin{array}{l} E \rightarrow E + E \\ E \rightarrow E * E \\ E \rightarrow id \end{array} \right\}, S = \{E\}$$



* Simplification of CFG :-

All the grammar are not always optimized. That means grammar may consists of extra symbols (Non terminal).

* Reduction of grammar by removing useless symbols



Properties

- (i) Each variable & terminal of G appear in the derivation of some word in L .
- (ii) There should not be any production as $X \rightarrow Y$ where X & Y are non-terminals.
- (iii) If ϵ is not in language L then there need not be the production $X \rightarrow \epsilon$.

* Types of Simplification :-

(1) Useless Symbols - A symbol is useless if -

It never produces a terminal string or
It is not reachable from start symbol.

(a) Non-generating Symbol
cannot produce any terminal

$A \rightarrow AB$ (useless)
 $B \rightarrow A$

(b) Unreachable Symbols
Cannot be reached from start symbol

$S \rightarrow AA$
 $A \rightarrow a$
 $B \rightarrow B$ (unreachable)

(2) Null Production (ε)

$A \rightarrow \epsilon$ (nullable)

- causes ambiguity
- Not allowed in CNF

$S \rightarrow AB \Rightarrow S \rightarrow AB | \epsilon$

$A \rightarrow A | \epsilon$ $A \rightarrow a$

$B \rightarrow b$ $B \rightarrow b$

(3) Unit Production

$A \rightarrow B$

- No direct contribution to terminal
- Increase unnecessary steps.

Ex Grammar

$S \rightarrow AB | a$

$A \rightarrow \epsilon$

$B \rightarrow b$

$C \rightarrow c$

Remove ϵ

$S \rightarrow AB | B | a$

$B \rightarrow b$

($C \rightarrow c$ Not required)

* Sentential Form:-

A sentential form is any string obtained during derivation. It may contain:-

- Terminal (a, b)
- Variable (S, A , etc)

$S \xRightarrow{*} \alpha$

α is sentential form.

$\alpha \in (V \cup T)^*$

E_1

$$S \rightarrow aSb \mid \epsilon$$

$$S \Rightarrow aSb$$

$$\Downarrow$$

$$aaSbb$$

$$\Downarrow$$

$$aabb \rightarrow \text{sentence}$$

} Sentential forms
(only terminals)

Left sentential form $\Rightarrow$ Obtained by leftmost derivation.

Right sentential form $\Rightarrow$ obtained by RMDT

Bottom-up parsing	Top-down parsing	Feature
Starts from root node and expands it to reach leaf nodes (bottom-up)	Starts from root node and expands it to reach leaf nodes (top-down)	Definition
Input string is given at the root node and is expanded to reach leaf nodes (bottom-up)	Input string is given at the root node and is expanded to reach leaf nodes (top-down)	Direction
Leftmost derivation (LR)	Rightmost derivation (RL)	Derivation
Complex	Simple	Complexity
Not required to know the whole string for parsing	Also known as with backtracking	Backtracking
Less powerful	More powerful	Power

* Parsing : process to checking whether a string belongs to a grammar & constructing its parse tree/derivation.

Ex Grammar - rules of language

Parsing - checking if sentence is correct.

⇒ Types of Parsing

i) Top-Down Parsing

ii) Bottom-Up Parsing

Feature	Top-Down Parsing	Bottom up Parsing
Definition	Starts from the start symbol (S) & tries to generate the input string	Starts from the input string & tries to reduce it to the start symbol (S)
Direction	Root → leaves	Leaves to Root
Derivation used	Leftmost derivation	Rightmost derivation (Reverse)
Approach	Expand	Reduced
Complexity	Simple	Complex
Backtracking	May require. (with also known as with backtracking)	Not required Also known as without backtracking.
Power	Less powerful	More powerful
Ex	LL (1)	LR, Shift Reduc.

Parsing

Top-down Parsing

• Recursive descent parsing

Bruteforce method

[with Backtracking]

• Non recursive descent parsing

[without back-tracking]

Bottom-up Parsing

LR Parsing

→ LR(1)

→ SLR(1)

→ LALR(1)

→ CLR(1)

Operator precedence parsing

• Recursive Descent parsing - uses backtracking.

→ Brute force approach.

→ Simple but inefficient.

• Non-Recursive Descent parsing - no backtracking

→ Uses parsing table.

→ Faster & efficient.

• LR parsing - Most powerful works for large grammar.

Types

LR(1), SLR(1), LALR(1), CLR(1)

• Operator Precedence parsing

Used for operator grammar

Condition → No ϵ production

→ no two adjacent non-terminals.

Operator Grammar

An operator grammar is a specific type of CFG with two main properties -

→ No ϵ production.

→ No two adjacent non terminals at right side.

* Chomsky Normal Form (CNF)

A CFM is in CNF if every production is of form (Standard form of grammar)

$$A \rightarrow BC \text{ or}$$

$$A \rightarrow a$$

Rules Right side of production Rule, only Two variable or one terminal are allowed.

ϵ is allowed only for $S \rightarrow \epsilon$

Ex $A \rightarrow aBX$

$$A \rightarrow BACX$$

$$A \rightarrow SX \text{ (only start symbol)}$$

$$A \rightarrow BX$$

It make parsing easier.

Q Conversion of CFM in CNF

$$S \rightarrow dAB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$T_1 \rightarrow a$$

$$S \rightarrow T_1AB$$

So

$$S \rightarrow T_1X_1$$

$$X_1 \rightarrow AB$$

$$S \rightarrow T_1X_1$$

$$T_1 \rightarrow a$$

$$X_1 \rightarrow AB$$

$$A \rightarrow a$$

$$B \rightarrow b$$

(1) Remove ϵ

(2) Remove unit state

(3) Remove useless

Symbol

(4) Convert Proper form

* Greibach Normal Form (GNF)

A CFG is in GNF if every production is of the form

$$A \rightarrow a\alpha$$

where a = terminal

α = strings of variable

Ex: $A \rightarrow aAB$

$A \rightarrow a$

$A \rightarrow AB$ X

$A \rightarrow Ba$ X

$A \rightarrow \epsilon$ ✓

not in $S \rightarrow \epsilon$

$A \rightarrow aB$ ✓

$A \rightarrow BC$ X

* Start with terminal)

Uses Top Down parsing.

Ex

$S \rightarrow AB$

$A \rightarrow a$

$B \rightarrow b$

$\Rightarrow S \rightarrow aB$

replace A

$S \rightarrow ab$

replace B

So

$S \rightarrow ab$

$A \rightarrow a$

$B \rightarrow b$

Difference

Feature	CNF (RMDT)	GNF (LMDT)
Rule	$A \rightarrow BC, A \rightarrow a$	$A \rightarrow a\alpha$
Start Symbol	Variable / Terminal	Terminal only
Parsing	Bottom-up parsing	Top-Down parsing
Conversion	Easier	Harder
Left recursion	Allowed	Not Allowed
Use	CYK	Predictive parsing

Problems Related to CNF & GNF

Q Convert the Given CFG to CNF

$$S \rightarrow aSa \mid bSb \mid a \mid b$$

Solⁿ

$$S \rightarrow ASA$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$S \rightarrow BSB$$

Rule 1

$$S \rightarrow a$$

$$\text{to } A \rightarrow a$$

replaced

Rule 2 Only two Non-terminal at right side

$$A_1 \rightarrow AS$$

(New variable)

$$B_1 \rightarrow BS$$

So Now

$$S \rightarrow A_1A$$

$$S \rightarrow B_1B$$

$$A \rightarrow a$$

$$B \rightarrow b$$

$$A_1 \rightarrow AS$$

$$B_1 \rightarrow BS$$

$$S \rightarrow a$$

$$S \rightarrow b$$

Q Consider $G = (\{S, A\}, \{a, b\}, P, S)$

while P consists of

$$S \rightarrow aAS \mid a$$

$$A \rightarrow SbA \mid SS \mid ba$$

convert into its eq. CNF.

$$S \rightarrow aAS$$

$$A_1 \rightarrow a$$

$$S_1 \rightarrow R_1AS$$

$$R_2 \rightarrow AS$$

$$S \rightarrow a$$

$$A \rightarrow SbA$$

$$\checkmark S \rightarrow R_1 R_2 \quad \checkmark S \rightarrow a$$

$$R_1 \rightarrow A \quad \text{for } A \rightarrow SR_3 A$$

$$R_2 \rightarrow A_1 S \quad R_3 \rightarrow b$$

$$A \rightarrow SR_3 A$$

$$R_4 \rightarrow R_3 A$$

$$\checkmark A \rightarrow SR_4$$

$$\text{in } \checkmark A \rightarrow SS$$

So finally.

$$S \rightarrow R_1 R_2$$

$$S \rightarrow a$$

$$A \rightarrow SR_4$$

$$R_4 \rightarrow R_3 A$$

$$R_3 \rightarrow b$$

$$R_1 \rightarrow a$$

$$R_2 \rightarrow A_1 S$$

$$\cancel{A \rightarrow SR_1}$$

$$A \rightarrow SS$$

$$A \rightarrow R_1 R_3 \quad \underline{\underline{A}}$$

Q. Convert the given GFG to CNF

Consider $V = \{v, T, P, S\}$

$V = \{S, A, B\}$

$T = \{a, b\}$

P consist of

$S \rightarrow aB$

$S \rightarrow bA$

$A \rightarrow a$

$A \rightarrow aS$

$A \rightarrow bAA$

$B \rightarrow b$

$B \rightarrow bS$

$B \rightarrow aBB$

Soln

for

$S \rightarrow aB$

$R_1 \rightarrow a$

$\checkmark S \rightarrow R_1 B$

$\checkmark A \rightarrow a \Rightarrow R_1 \rightarrow a$

$A \rightarrow aS$

$\checkmark A \rightarrow R_1 S$

$A \rightarrow \underline{b} \underline{AA}$

$\checkmark A \rightarrow R_2 R_3$

$\checkmark R_3 \rightarrow AA$

$S \rightarrow bA$

$R_2 \rightarrow b$

$\checkmark S \rightarrow R_2 A$

$\checkmark B \rightarrow b \Rightarrow R_2 \rightarrow b$

$B \rightarrow bS$

$\checkmark B \rightarrow R_2 S$

$B \rightarrow \underline{a} \underline{BB}$

$\checkmark R_4 \rightarrow BB$

$\checkmark B \rightarrow R, R_4$

So CNF are

$S \rightarrow R_1 B / R_2 A$

$A \rightarrow R_1 S / R_2 R_3 / a$

$B \rightarrow R_2 S / R_1 R_4 / b$

$R_1 \rightarrow a$

$R_3 \rightarrow AA$

$R_2 \rightarrow b$

$R_4 \rightarrow BB$